



College of Computer Science and Engineering
Department of Computer Science and Artificial
Intelligence
CCAI-312: Pattern Recognition
CCAI312 Group Project
(Fall 2025)

	Max Score	Student 1 score	Student 2 score	Student 3 score	Student 4 score
--	----------------------	----------------------------	----------------------------	----------------------------	----------------------------

PLO C4 / CLO 3.1 / SO 7	10				
--------------------------------	-----------	--	--	--	--

Students' names	Students' IDs
Hutaf Alkhudaydi	2312439
Minnah karam	2317478
Maha Mahdi	2317197
Baraah Khoja	2310445

PLO/CLO	SO
PLO C4 (CLO 3.1): Discover and apply new knowledge as needed, using appropriate learning strategies	SO 7: An ability to acquire and apply new knowledge as needed, using appropriate learning strategies.

Total					
--------------	--	--	--	--	--

TabTransformer: A Deep Learning Architecture for Bank Deposit Subscription Prediction

Minnah Karam , Hutaf Alkhudaydi , Maha mahdi , Baraah khoja
College of Computer Science & Engineering, University of Jeddah, Jeddah, Saudi Arabia
5 December 2025

ABSTRACT

This project presents a machine-learning framework for predicting customer subscription to term deposits using the Bank Marketing Dataset. Three models were evaluated: Logistic Regression, Decision Tree, and the TabTransformer, a deep-learning architecture designed for tabular data. After preprocessing with label encoding and standardization, models were trained and compared using accuracy, precision, recall, F1-score, ROC-AUC, and Average Precision. Results show that the TabTransformer significantly outperforms classical models due to its ability to learn contextual and high-order interactions among categorical features, demonstrating the advantages of transformer-based deep learning for structured banking data.

1. INTRODUCTION

Predicting customer subscription to term deposits is an essential task in financial marketing. Banks rely on predictive analytics to optimize campaign targeting, reduce costs, and increase conversion rates. Machine-learning models enable the identification of behavioral patterns based on demographic information, historical interactions, and financial attributes.

This project uses the **Bank Marketing Dataset**, collected from direct marketing campaigns of a Portuguese banking institution.

Traditional models such as Logistic Regression and Decision Trees can capture basic decision boundaries but struggle with complex interactions among categorical attributes. To address this, we implement the **TabTransformer**, a transformer-based architecture designed specifically for tabular data with multiple categorical variables. The goal of this study is to compare classical baselines with both a default and a hyperparameter-tuned TabTransformer model.

2. RELATED WORK

Recent research on tabular data modeling has increasingly shifted toward transformer-based and deep learning architectures, reflecting the growing interest in leveraging attention mechanisms and contextual embeddings to handle mixed numerical and categorical features. A comprehensive survey published in 2023 reviewed multiple transformer-based encoders and highlighted their potential as strong alternatives to classical models for tabular classification tasks.[1]

In 2024, a study introduced a multimodal transformer architecture for tabular data that integrates numerical, categorical, and textual attributes, demonstrating notable improvements in classification accuracy when richer feature interactions are present. [2] More recently, a 2025 work explored data-efficient transformerbased models for tabular classification under limited data conditions. The findings showed that with appropriate embedding strategies and attention mechanisms, deep models can rival or surpass gradient-boosting methods on several structured datasets. [3].

Additionally, a large-scale empirical analysis published in 2025 examined the performance of deep tabular learning across more than 300 datasets. The study concluded that while classical models remain competitive, transformer-based architectures achieve superior performance on datasets with complex feature dependencies and heterogeneous feature types.[4] Together, these recent contributions provide strong motivation for applying transformer-based architectures—such as the TabTransformer—to financial prediction tasks. They also support the comparative evaluation conducted in this project between deep learning models and traditional machine-learning baselines.

3. Data Description

The dataset contains **11,162 rows and 16 input features**, the dataset includes both numerical (age, balance, duration, etc.) and categorical attributes (job, education, marital status, contact type, etc.). The target variable indicates whether a customer subscribed to a term deposit.

During the data analysis, we observed several outliers in numerical features such as *balance*, *duration*, and *campaign*.

We did not remove them because these values represent real customer behavior rather than data-entry errors. Additionally, our model is based on Deep Learning.

The most relevant features include the customer's their previous marketing interaction history (duration, previous, poutcome), and key campaign-related attributes such as month and contact type.

(TabTransformer), which is robust to high-variance data. Removing these outliers could eliminate important information that helps the model learn meaningful patterns.

Preprocessing included:

- Verifying the absence of missing values
- Label Encoding for categorical variables
- Standardization of numerical features using StandardScaler
- Stratified splitting into training, validation, and test sets (to preserve class balance)

The dataset remains mildly imbalanced, with slightly more negative samples (no subscription) than positive ones.

4. Methodology

4.1 Overview

The workflow includes:

1. Preprocessing numeric and categorical features
2. Training baseline models
3. Implementing a default TabTransformer
4. Hyperparameter tuning using randomized search
5. Evaluating all models using multiple metrics

Baseline classifiers (Logistic Regression and Decision Tree) serve as comparison points for the transformer-based models.

4.2 TabTransformer Architecture

The TabTransformer processes categorical features using embeddings and transformer encoder blocks, while numerical features bypass the transformer and are concatenated later before classification.

4.2.1 Categorical Embedding Pipeline

Label-encoded categories are mapped to trainable embedding vectors.

Embeddings are passed through stacked Transformer Encoder Blocks containing:

- Multi-Head Self-Attention
- Feed-Forward Networks (FFN)
- Layer Normalization
- Residual connections

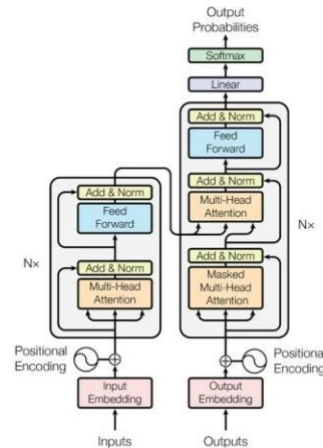


Figure 1: TabTransformer Encoder Block

4.2.2 Multi-Head Self-Attention

Self-attention models pairwise relationships between categorical tokens, enabling the model to learn feature interactions that traditional methods cannot capture.

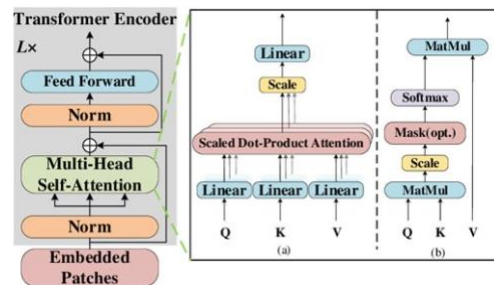


Figure 2: Multi-Head Self-Attention Mechanism

4.2.3 Numerical Feature Pathway

Numerical features (after standardization) are concatenated with the transformer output prior to classification.

4.2.4 Classification Head

The final vector is passed to an MLP consisting of:

- Dense(64, ReLU)

- Dense(32, ReLU)
- Dense(1, Sigmoid)

This head was **not included in the tuning search** and remained fixed.

4.2.5 Loss Function

All models in this project were trained using the **Binary CrossEntropy (BCE)** loss function, which is the standard objective for binary classification tasks. BCE measures how well the predicted probabilities match the true class labels and provides sharper gradients than regression-based losses, making it more suitable for imbalanced or probabilistic classification problems.

The BCE loss is defined mathematically as: $L = -$

$$(1/N) \sum_{i=1}^N [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Where:

- y_i = true label (0 or 1)
- $\hat{y}_i$ = predicted probability
- N = number of samples

4.3 Hyperparameter Tuning A

randomized search explored:

- $d_model \in \{16, 32, 64\}$
- $num_heads \in \{2, 4, 8\}$
- $num_blocks \in \{1, 2, 3\}$
- $ff_dim \in \{32, 64, 128\}$

Each trial was trained with EarlyStopping and ReduceLROnPlateau.

The **best configuration** (based on validation accuracy) was: d_model

= 16, num_heads = 8, num_blocks = 2, ff_dim = 128

5. EXPERIMENTS

This section describes the training procedure, preprocessing pipeline, and evaluation methods used across all models.

5.1 Data Splitting

The dataset was divided using a **stratified 70/15/15 split**, ensuring that the positive and negative classes remained balanced across training, validation, and testing sets. This helps maintain consistent evaluation and prevents shifts in class distribution from affecting performance.

5.2 Training Configuration

All models—including baselines and TabTransformer variants—were trained on the same preprocessed dataset. Numerical features were standardized, and categorical features were label-encoded before being passed into embedding layers.

The TabTransformer models were trained for **15 epochs** with:

- **Batch size:** 32
- **Optimizer:** Adam • **Loss function:** Binary Cross-Entropy
- **Callbacks:**
 - *EarlyStopping*(patience=2), *ReduceLROnPlateau* (factor=0.5, patience=1) during hyperparameter tuning
 - *EarlyStopping* (patience=3), *ReduceLROnPlateau* (patience=2) during training of the default and tuned models.

5.3 Baseline Models

Two baseline models were included for comparison:

- Logistic Regression
- Decision Tree Classifier

Both models were trained on the same encoded + scaled input to ensure fairness in comparison.

5.4 Hyperparameter Tuning (Methodology Reference)

A randomized hyperparameter search (described in Section 4.3) was used to explore variations in embedding size, attention heads, transformer blocks, and feed-forward dimension. Only the **best configuration** from the search was evaluated and compared to the default model.

5.5 Evaluation Metrics

Models were evaluated using:

- **Accuracy**
- **Precision, Recall, Macro-F1**
- **ROC-AUC**
- **Average Precision (AP)**
- **Confusion Matrix**
- **ROC & PR Curves**

6. RESULTS AND DISCUSSION

This section presents the performance of the baseline models, the default TabTransformer, and the tuned TabTransformer. Each result is followed by a short discussion to interpret model behavior and explain performance differences.

6.1 Baseline Models Performance

Table 1 summarizes the performance of Logistic Regression and Decision Tree on the test set.

Model	Accuracy	ROC-AUC	AP
Logistic Regression	0.7922	0.8610	0.8315
Decision Tree	0.7875	0.8463	0.7752

Both baseline models achieve moderate performance, with Logistic Regression performing slightly better. However, both models struggle to capture the complex nonlinear interactions present in the dataset.

6.2Default TabTransformer Results

The default TabTransformer significantly improves performance over baseline models.

Table 2. Default TabTransformer Performance

Metric	Value
Accuracy	0.8442
ROC-AUC	0.9123
AP	0.8719

Class	Precision	Recall	F1
0	0.9101	0.7809	0.8406
1	0.7900	0.9144	0.8476

Figure 3.Training Curves for Default TabTransformer

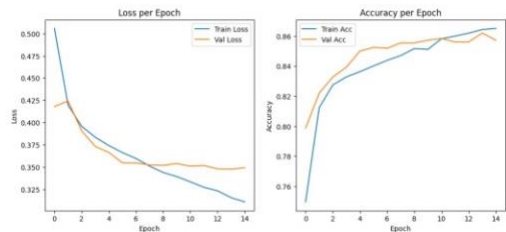


Figure 3. Loss and accuracy per epoch for the default TabTransformer.

Figure 4. Confusion Matrix for Default TabTransformer

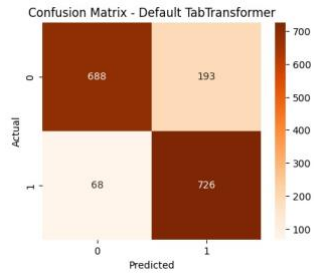


Figure 4. Confusion matrix showing prediction distribution for both classes.

The default model achieves strong performance, especially in detecting positive class samples (recall = 0.9144). The training curves indicate stable convergence without overfitting, confirming that early stopping and learning rate reduction were effective.

6.3 Tuned TabTransformer Results

Randomized search selected the following configuration as the best:

d_model = 16, num_heads = 8 , num_blocks = 2, ff_dim = 128

Table 3. Tuned TabTransformer Performance

Metric	Value
Accuracy	0.8400
ROC-AUC AP	0.9177 0.8888

Class	Precision	Recall	F1
0	0.9060	0.7764	0.8362
1	0.7859	0.9106	0.8436

Figure 5.Training Curves for Tuned TabTransformer

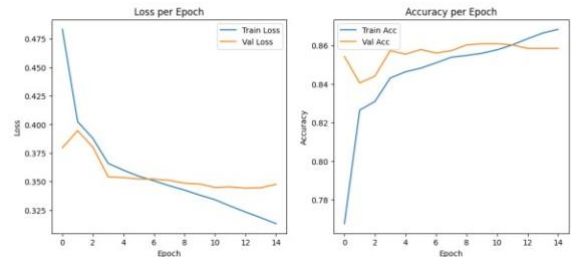


Figure 5. Loss and accuracy per epoch for the tuned TabTransformer.

Figure 6. Confusion Matrix for Tuned TabTransformer

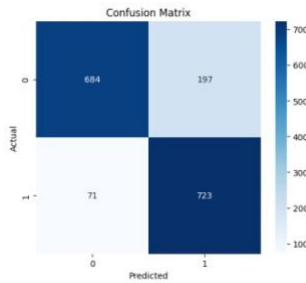


Figure 6. Confusion matrix of the tuned TabTransformer.

Although tuning improved ROC-AUC slightly, accuracy and macro-F1 decreased. This indicates that the tuned architecture improves ranking ability but becomes less calibrated at the classification threshold.

6.4 Comparative ROC and PR Curve

To further evaluate probability ranking quality, both ROC and Precision–Recall (PR) curves were generated by comparing the baseline models with the final tuned TabTransformer. While ROC curves measure overall ranking performance across thresholds, the PR curve is more sensitive to class imbalance and highlights precision–recall trade-offs for the positive class.

Figure 7. ROC Curve Comparison: Baseline vs. Tuned TabTransformer

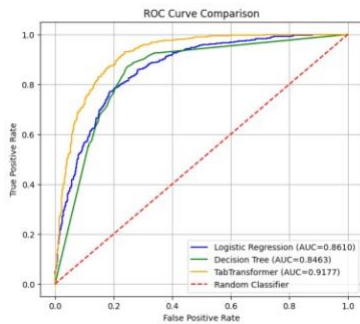


Figure 7. The transformer model shows a consistently higher true-positive rate across all thresholds.

Figure 8. PR Curve Comparison: Baseline vs. Tuned TabTransformer

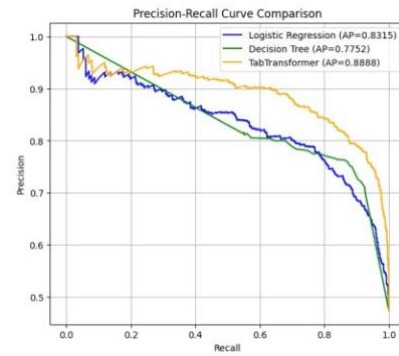


Figure 8. Precision–Recall curve comparison. The AP score reported in the results corresponds to the area under this curve.

The ROC comparison highlights the superiority of attention-based models for tabular data, as the TabTransformer achieves smoother and more stable ranking behavior than both Logistic Regression and the Decision Tree. The Precision–Recall comparison further confirms this advantage, showing that the TabTransformer maintains higher precision across different recall levels, especially for the positive class. This indicates that the transformer-based architecture is better at identifying deposit subscribers, even under class imbalance.

6.5 Overall Comparison

Table 4. Performance Summary of All Models

Model	Accuracy	Macro-F1	ROC-AUC	AP
Logistic Regression	0.7922	0.7900	0.8610	0.8315
Decision Tree	0.7875	0.7900	0.8463	0.7752
Default TabTransformer	0.8442	0.8441	0.9123	0.8719
Tuned TabTransformer	0.8400	0.8399	0.9177	0.8888

The summary table shows that TabTransformer models significantly outperform the classical baselines across all evaluation metrics. The Default TabTransformer achieves the highest accuracy and macro-F1 scores, demonstrating strong performance at the 0.5 classification threshold. In contrast, the Tuned TabTransformer achieves the best ROC-AUC and AP values, indicating superior ranking ability and improved detection of the positive class across different probability thresholds. These results highlight that transformer-based architectures better capture complex feature interactions in tabular banking data compared to traditional machine-learning models.

7. Conclusion

This project evaluated classical machine-learning models and transformer-based architectures for predicting customer subscription to bank term deposits. The results demonstrated that

the TabTransformer significantly performs better than Logistic Regression and Decision Tree models across multiple evaluation metrics. The default TabTransformer achieved the highest accuracy and macro-F1 scores, while the tuned version obtained the best ROC-AUC and Average Precision, indicating superior ranking performance and stronger detection capability for the positive class. These findings highlight the effectiveness of attention mechanisms in capturing complex feature interactions within tabular banking data. Future improvements may include exploring larger embedding dimensions, additional regularization strategies, or pretraining techniques to further enhance model stability and performance.

8. REFERENCES

- [1] Transformers for Tabular Data Representation: A Survey. Transactions of the ACL, 2023.
- [2] Revisiting Multimodal Transformers for Tabular Data with Text Fields. Findings of ACL, 2024.
- [3] Data-Efficient Tabular Classification with TransformerBased Models. WebMedia Conference, 2025.
- [4] A Closer Look at Deep Learning Methods on Tabular Data. arXiv preprint, 2025
- [5] Vaswani, Ashish, et al. "Attention Is All You Need." Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [6] Dosovitskiy, Alexey, et al. "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale." International Conference on Learning Representations (ICLR), 2021.

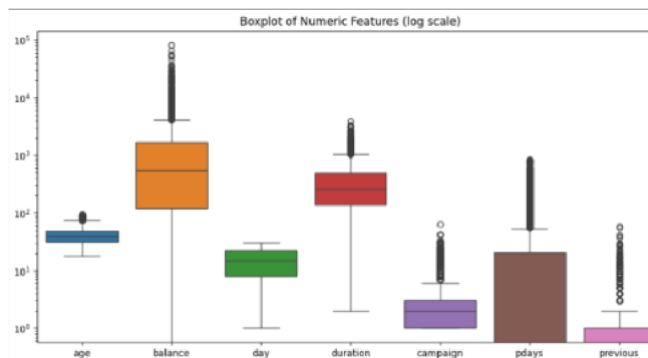
10 Appendix A: Exploratory Data Analysis (EDA) Figures

Figure A1. Deposit Target Distribution



This figure shows the distribution of the target classes. The “no deposit” class is slightly more frequent than the “yes” class, indicating a mild class imbalance in the dataset.

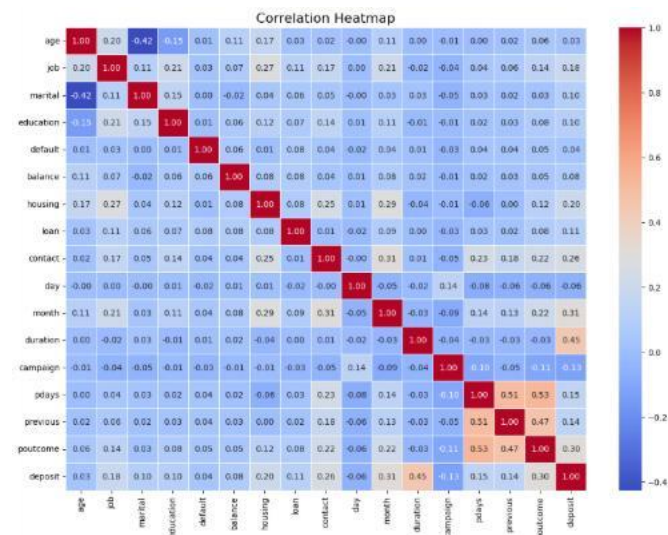
Figure A2. Boxplot of Numerical Features After Standardization



This figure displays the distribution of the standardized numerical features and reveals noticeable variability and several outliers across different attributes—particularly in *balance*, *duration*, *pdays*, and

previous. These variations suggest that some features are highly skewed or contain extreme values, which can influence model learning and contribute to differences in predictive performance. **Appendix B— TabTransformer Model Architecture**

Figure A3. Correlation Heatmap Using Target Encoding



The heatmap provides an approximate measure of how features relate to the target variable after applying target encoding for visualization purposes.

```
# Build TabTransformer without Stack

def transformer_block(x, head, dim, ff_dim):
    attn = layers.MultiHeadAttention(num_heads=head, key_dim=dim)(x, x)
    x = layers.Add()([x, attn])
    x = layers.LayerNormalization()(x)

    # FNN
    ff = layers.Dense(ff_dim, activation="relu")(x)
    ff = layers.Dense(dim)(ff)

    x = layers.Add()([x, ff])
    x = layers.LayerNormalization()(x)
    return x

def build_tabtransformer(
    num_numeric,
    num_categories,
    vocab_sizes,
    dim=32,
    heads=4,
    blocks=2,
    ff_dim=128,
):
```

```

# Numeric features input
numeric_in = layers.Input(shape=(num_numeric,), name="numeric_inputs")

# Embeddings for categorical
cat_inputs = []
cat_embeds = []

for i, vocab in enumerate(vocab_sizes):
    inp = layers.Input(shape=(1,), name=f"cat{i}")
    emb = layers.Embedding(input_dim=vocab, output_dim=dim)(inp)
    emb = layers.Reshape((dim,))(emb)
    cat_inputs.append(inp)
    cat_embeds.append(emb)

# Stack embeddings as tokens: (batch, num_categories, dim)
x = layers.Lambda(lambda t: tf.stack(t, axis=1))(cat_embeds)

# Transformer blocks using ff_dim
for _ in range(blocks):
    x = transformer_block(x, heads, dim, ff_dim)

# Flatten tokens
x = layers.Flatten()(x)

# Concatenate with numeric
combined = layers.Concatenate()([x, numeric_in])

# MLP head
out = layers.Dense(64, activation="relu")(combined)
out = layers.Dense(32, activation="relu")(out)
out = layers.Dense(1, activation="sigmoid")(out)

return Model(inputs=[numeric_in] + cat_inputs, outputs=out)

```

Appendix C — Training Procedure

```

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import matplotlib.pyplot as plt

# Setting up callbacks
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

lr_reducer = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=2,
    min_lr=1e-5
)

# Setting up vocab_sizes for each categorical column
vocab_sizes = [df[col].nunique() + 1 for col in categorical_cols]

```

```

# Building the model
model = build_tabtransformer(
    num_numeric=len(numeric_cols),
    num_categories=len(categorical_cols),
    vocab_sizes=vocab_sizes,
    dim=32,
    heads=4,
    blocks=2,
    ff_dim=128,
)

model.compile(
    loss="binary_crossentropy",
    optimizer="adam",
    metrics=["accuracy"]
)

# Preparing the inputs
train_inputs = {"numeric_inputs": X_train_num}
val_inputs = {"numeric_inputs": X_val_num}
test_inputs = {"numeric_inputs": X_test_num}

for i, col in enumerate(categorical_cols):
    train_inputs[f"cat{i}"] = X_train_cat[col]
    val_inputs[f"cat{i}"] = X_val_cat[col]
    test_inputs[f"cat{i}"] = X_test_cat[col]

# Training the model
history = model.fit(
    train_inputs, y_train,
    validation_data=(val_inputs, y_val),
    epochs=15,
    batch_size=32,
    callbacks=[early_stop, lr_reducer]
)

# Model probabilities before tuning on the test set
y_pred_proba_default = model.predict(test_inputs).flatten()

```

Appendix D — Model Evaluation

```

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Converting probabilities to 0/1 class labels
y_pred_default = (y_pred_proba_default > 0.5).astype(int)

print("Accuracy (Default TabTransformer):", accuracy_score(y_test, y_pred_default))
print("Classification Report (Default TabTransformer):")
print(classification_report(y_test, y_pred_default, digits=4))

cm_default = confusion_matrix(y_test, y_pred_default)
plt.figure(figsize=(5,4))
sns.heatmap(cm_default, annot=True, fmt='d',
            cmap='Oranges', xticklabels=[0,1], yticklabels=[0,1])
plt.title("Confusion Matrix - Default TabTransformer")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```